

1. vector — Dynamic Array

What it does	Memory	Does NOT support
Contiguous resizable array. Random access in $O(1)$. Doubles capacity on overflow.	$O(n)$	No $O(1)$ insert/delete in the middle.

```
#include <vector>
// empty vector v(n, 0); // n zeros
```

Operation	C++ Function	Time	Notes
Access element	<code>v[i] / v.at(i)</code>	$O(1)$	<code>at()</code> does bounds check
Push to back	<code>v.push_back(x)</code>	$O(1)$ amort	Rarely $O(n)$ on resize
Pop from back	<code>v.pop_back()</code>	$O(1)$	
Insert at position	<code>v.insert(it, x)</code>	$O(n)$	Shifts all elements right
Erase at position	<code>v.erase(it)</code>	$O(n)$	Shifts all elements left
Size / Empty	<code>v.size() / v.empty()</code>	$O(1)$	
Clear	<code>v.clear()</code>	$O(n)$	Destroys all elements
Sort	<code>sort(v.begin(), v.end())</code>	$O(n \log n)$	<code>std::sort</code>
Binary search	<code>lower_bound / upper_bound</code>	$O(\log n)$	Only on sorted vector
Front / Back	<code>v.front() / v.back()</code>	$O(1)$	

2. stack — LIFO Adapter

What it does	Memory	Does NOT support
Last-In First-Out. Only the top is accessible. Built on deque by default.	$O(n)$	No iteration, no random access, no search.

```
#include <stack>
```

Operation	C++ Function	Time	Notes
Push	<code>st.push(x)</code>	$O(1)$	
Pop	<code>st.pop()</code>	$O(1)$	Does NOT return value — call <code>top()</code> first
Top	<code>st.top()</code>	$O(1)$	Undefined on empty stack
Size/Empty	<code>st.size() / st.empty()</code>	$O(1)$	

3. queue — FIFO Adapter

What it does	Memory	Does NOT support
--------------	--------	------------------

First-In First-Out. Push to back, pop from front.	$O(n)$	No random access, no iteration, no search.
---	--------	--

```
#include queue q;
```

Operation	C++ Function	Time	Notes
Push to back	<code>q.push(x)</code>	$O(1)$	
Pop from front	<code>q.pop()</code>	$O(1)$	Does NOT return value
Front	<code>q.front()</code>	$O(1)$	Oldest element
Back	<code>q.back()</code>	$O(1)$	Newest element
Size / Empty	<code>q.size() / q.empty()</code>	$O(1)$	

4. deque — Double-Ended Queue

What it does	Memory	Does NOT support
Like vector but supports $O(1)$ push/pop at both ends. Not contiguous in memory.	$O(n)$	Slower random access than vector (cache misses). No $O(1)$ middle insert.

```
#include deque dq;
```

Operation	C++ Function	Time	Notes
Push front	<code>dq.push_front(x)</code>	$O(1)$	
Push back	<code>dq.push_back(x)</code>	$O(1)$	
Pop front	<code>dq.pop_front()</code>	$O(1)$	
Pop back	<code>dq.pop_back()</code>	$O(1)$	
Random access	<code>dq[i]</code>	$O(1)$	Slower than vector in practice
Insert middle	<code>dq.insert(it, x)</code>	$O(n)$	
Size / Empty	<code>dq.size() / dq.empty()</code>	$O(1)$	

5. priority_queue — Heap

What it does	Memory	Does NOT support
Binary heap. Max-heap by default: <code>top()</code> is always the LARGEST element. Use <code>greater</code> for min-heap (<code>top</code> = smallest). Does NOT fully sort — only guarantees the top.	$O(n)$	No arbitrary delete. No search. No update-priority.

```
#include priority_queue pq; // max-heap priority_queue, greater> pq; // min-heap
```

Operation	C++ Function	Time	Notes
Push	<code>pq.push(x)</code>	$O(\log n)$	Sifts up the tree
Top	<code>pq.top()</code>	$O(1)$	Max (or min) element
Pop top	<code>pq.pop()</code>	$O(\log n)$	Sifts down after removal
Size / Empty	<code>pq.size() / pq.empty()</code>	$O(1)$	

Operation	C++ Function	Time	Notes
Build from array	<code>priority_queue pq(v.begin(), v.end())</code>	$O(n)$	Heapify — faster than n pushes

Memory detail: stores exactly n elements in a plain array — $4n$ bytes for `int`. Much lighter than `set/map` ($\sim 40n$ bytes) because there are no pointers.

6. set — Sorted Unique Elements (Red-Black Tree)

What it does	Memory	Does NOT support
Stores all elements in SORTED order, unique. Every operation traverses a balanced BST of height $O(\log n)$. Unlike heap, ALL elements are ordered — not just the top.	$O(n)$ ($\sim 40n$ bytes)	No random access (no <code>operator[]</code>). No duplicates (use <code>multiset</code>).

```
#include <set>
set s;
```

Operation	C++ Function	Time	Notes
Insert	<code>s.insert(x)</code>	$O(\log n)$	
Delete	<code>s.erase(x)</code> or <code>s.erase(it)</code>	$O(\log n)$	By value or iterator
Search	<code>s.find(x)</code>	$O(\log n)$	Returns <code>end()</code> if not found
Contains	<code>s.count(x)</code> — returns 0 or 1	$O(\log n)$	
Min element	<code>*s.begin()</code>	$O(1)$	Leftmost in tree
Max element	<code>*s.rbegin()</code>	$O(1)$	Rightmost in tree
Lower bound	<code>s.lower_bound(x)</code>	$O(\log n)$	First element $\geq x$
Upper bound	<code>s.upper_bound(x)</code>	$O(\log n)$	First element $> x$
Size / Empty	<code>s.size()</code> / <code>s.empty()</code>	$O(1)$	
Iterate in order	<code>for(auto x : s)</code>	$O(n)$	Always sorted

7. multiset — Sorted with Duplicates

What it does	Memory	Does NOT support
Same as <code>set</code> but allows duplicate values. <code>count(x)</code> returns the number of occurrences.	$O(n)$ ($\sim 40n$ bytes)	<code>erase(x)</code> removes ALL copies of x . Use <code>erase(s.find(x))</code> to remove just one copy.

```
#include <multiset>
multiset ms;
```

Operation	C++ Function	Time	Notes
Insert	<code>ms.insert(x)</code>	$O(\log n)$	Duplicates allowed
Erase all copies	<code>ms.erase(x)</code>	$O(\log n + \text{count})$	Removes every x
Erase one copy	<code>ms.erase(ms.find(x))</code>	$O(\log n)$	Safe way to remove single
Count occurrences	<code>ms.count(x)</code>	$O(\log n + \text{count})$	

Operation	C++ Function	Time	Notes
Min / Max	<code>*ms.begin()</code> / <code>*ms.rbegin()</code>	$O(1)$	
Lower/Upper bound	<code>ms.lower_bound(x)</code>	$O(\log n)$	Same as set

8. map — Sorted Key-Value Store (Red-Black Tree)

What it does	Memory	Does NOT support
Stores key-value pairs sorted by key. Keys are unique. <code>operator[]</code> inserts a default value if key is absent — be careful.	$O(n)$ (~56n bytes)	No random index access. No duplicate keys (use <code>multimap</code>).

```
#include <map> mp;
```

Operation	C++ Function	Time	Notes
Insert / Update	<code>mp[key] = val</code>	$O(\log n)$	Creates key if absent!
Safe insert	<code>mp.insert({key, val})</code>	$O(\log n)$	Won't overwrite existing
Access	<code>mp[key]</code> or <code>mp.at(key)</code>	$O(\log n)$	<code>at()</code> throws if key missing
Delete	<code>mp.erase(key)</code>	$O(\log n)$	
Search	<code>mp.find(key)</code>	$O(\log n)$	Returns <code>end()</code> if not found
Contains	<code>mp.count(key)</code> — 0 or 1	$O(\log n)$	
Iterate in order	<code>for(auto& [k,v] : mp)</code>	$O(n)$	Sorted by key
Size / Empty	<code>mp.size()</code> / <code>mp.empty()</code>	$O(1)$	
Lower/Upper bound	<code>mp.lower_bound(key)</code>	$O(\log n)$	On keys

9. unordered_set — Hash Set

What it does	Memory	Does NOT support
Hash table. Average $O(1)$ for insert/search/delete. NO ordering — iteration order is arbitrary. Worst case $O(n)$ on hash collisions (rare but possible in contests with anti-hash tests).	$O(n)$ (~several $\times$ n bytes for buckets)	No sorted order. No <code>lower_bound/upper_bound</code> . Worst case $O(n)$ with bad hash.

```
#include <unordered_set> us;
```

Operation	C++ Function	Time	Notes
Insert	<code>us.insert(x)</code>	$O(1)$ avg	$O(n)$ worst case
Delete	<code>us.erase(x)</code>	$O(1)$ avg	
Search	<code>us.find(x)</code> / <code>us.count(x)</code>	$O(1)$ avg	
Size/Empty	<code>us.size()</code> / <code>us.empty()</code>	$O(1)$	

10. unordered_map — Hash Map

What it does	Memory	Does NOT support
Hash table for key-value pairs. Average $O(1)$ for all operations. No ordering.	$O(n)$ (~several $\times$ n bytes for buckets)	No sorted order. No <code>lower_bound</code> . Worst case $O(n)$ collisions.

```
#include unordered_map ump;
```

Operation	C++ Function	Time	Notes
Insert / Update	<code>ump[key] = val</code>	$O(1)$ avg	Creates key if absent
Access	<code>ump[key] / ump.at(key)</code>	$O(1)$ avg	<code>at()</code> throws if missing
Delete	<code>ump.erase(key)</code>	$O(1)$ avg	
Search	<code>ump.find(key)</code>	$O(1)$ avg	
Contains	<code>ump.count(key)</code>	$O(1)$ avg	
Size / Empty	<code>ump.size() / ump.empty()</code>	$O(1)$	

Quick Comparison

Container	Ordered?	Unique?	Insert	Delete	Search	Min/Max	Memory/elem
vector	index order	yes	$O(1)^*$	$O(n)$	$O(n)$	$O(n)$	4 B (int)
stack	LIFO	yes	$O(1)$	top only	—	—	4 B
queue	FIFO	yes	$O(1)$	front only	—	—	4 B
deque	index order	yes	$O(1)$ ends	$O(n)$ mid	$O(n)$	$O(n)$	~4 B
priority_queue	heap order	yes	$O(\log n)$	top only	—	$O(1)$ top	4 B
set	sorted	yes	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$	~40 B
multiset	sorted	NO	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$	~40 B
map	sorted	yes	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$ key	~56 B
unordered_set	NONE	yes	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	—	~varies
unordered_map	NONE	yes	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	—	~varies

** vector push_back is $O(1)$ amortised — occasionally $O(n)$ when capacity doubles. Memory per element for set/map/unordered containers is approximate and depends on allocator.*

11. string — Dynamic Character Array

What it does	Memory	Does NOT support
Contiguous, resizable array of char. Behaves like <code>vector<char></code> with text helpers. Short strings often live inline (SSO) with no heap allocation. Comparison is lexicographic, not by length.	$O(n)$	No $O(1)$ insert/erase in the middle. No built-in split. <code>find()</code> is naive, not KMP — $O(n \times m)$ worst case.

```
#include <string>    string s;    string s(n, 'a'); // n copies of 'a'
```

Operation	C++ Function	Time	Notes
Access char	<code>s[i]</code> / <code>s.at(i)</code>	$O(1)$	<code>at()</code> does bounds check
Append char	<code>s += c</code> / <code>s.push_back(c)</code>	$O(1)$ amort	Same growth as vector
Append string	<code>s += t</code> / <code>s.append(t)</code>	$O(m)$	m = length of t
Concatenate	<code>s + t</code>	$O(n+m)$	Allocates a new string — avoid in loops
Substring	<code>s.substr(i, len)</code>	$O(len)$	Copies — not a view
Find substring	<code>s.find(t)</code>	$O(n \times m)$ worst	Returns <code>string::npos</code> if absent
Compare	<code>s == t</code> / <code>s < t</code>	$O(n)$	Lexicographic order
Insert	<code>s.insert(i, t)</code>	$O(n)$	Shifts the tail right
Erase	<code>s.erase(i, len)</code>	$O(n)$	Shifts the tail left
Replace	<code>s.replace(i, len, t)</code>	$O(n)$	Length may change
Length	<code>s.size()</code> / <code>s.length()</code>	$O(1)$	Identical functions
Sort characters	<code>sort(s.begin(), s.end())</code>	$O(n \log n)$	Anagram check idiom
Reverse	<code>reverse(s.begin(), s.end())</code>	$O(n)$	In place
String to int	<code>stoi(s)</code> / <code>stoll(s)</code>	$O(n)$	Throws on bad input
Int to string	<code>to_string(x)</code>	$O(n)$	
Front / Back	<code>s.front()</code> / <code>s.back()</code>	$O(1)$	Undefined if empty
C-string	<code>s.c_str()</code>	$O(1)$	Null-terminated <code>const char*</code>

12. String Algorithms — Contest Toolkit

What it does	Memory	Does NOT support
Standard preprocessing techniques. Each trades $O(n)$ build time for fast queries afterwards. Reach for these when <code>s.find()</code> in a loop is $O(n^2)$ and too slow.	$O(n)$	Hashing is probabilistic — anti-hash tests exist. Use a random mod or double hashing on Codeforces.

Technique	Core Object	Time	Use it for
Prefix function (KMP)	<code>pi[i]</code> = longest proper border	$O(n)$	Pattern matching, period detection
Z-function	<code>z[i]</code> = <code>lcp(s, s[i..])</code>	$O(n)$	Matching, distinct-prefix counting
Polynomial hashing	<code>h[i]</code> , <code>pw[i]</code> arrays	$O(n)$ build, $O(1)$ query	Compare any two substrings
Trie	26-ary node tree	$O(len)$ per word	Prefix queries, XOR-max on bits
Suffix array + LCP	sorted suffix order	$O(n \log n)$	k -th substring, longest repeat
Manacher	palindrome radii	$O(n)$	All palindromic substrings
Aho-Corasick	trie + failure links	$O(\text{total} + \text{matches})$	Many patterns at once

13. Bitwise Operators

What it does	Memory	Does NOT support
Operate on the binary representation directly. All are single CPU instructions — effectively free compared with any container operation.	$O(1)$	Shifting by $\geq$ the type's width is undefined behaviour. Shifting a negative value left is undefined.

Operator	Expression	Meaning	Example (8-bit)
AND	<code>a & b</code>	1 only where both bits are 1	<code>1100 & 1010 = 1000</code>

Operator	Expression	Meaning	Example (8-bit)
OR	$a \mid b$	1 where either bit is 1	$1100 \mid 1010 = 1110$
XOR	$a \wedge b$	1 where the bits differ	$1100 \wedge 1010 = 0110$
NOT	$\sim a$	Flips every bit	$\sim 0000 = 1111$ (i.e. -1)
Left shift	$a \ll k$	Multiply by 2^k	$0011 \ll 2 = 1100$
Right shift	$a \gg k$	Divide by 2^k (floor)	$1100 \gg 2 = 0011$

14. Bit Manipulation — Idioms

What it does	Memory	Does NOT support
The standard vocabulary for treating an int as a set of up to 32 (or 64) elements. Underpins bitmask DP, subset enumeration and XOR-basis arguments.	$O(1)$ each	<code>__builtin_clz(0)</code> and <code>__builtin_ctz(0)</code> are undefined. Use <code>1LL << i</code> whenever $i \geq 31$, or the shift silently overflows.

```
// no header needed – operators are built in; __builtin_* are GCC/Clang intrinsics
```

Task	Expression	Time	Notes
Read i-th bit	$(x \gg i) \& 1$	$O(1)$	0-indexed from the LSB
Set i-th bit	$x \mid= (1 \ll i)$	$O(1)$	Use <code>1LL</code> for long long
Clear i-th bit	$x \&= \sim(1 \ll i)$	$O(1)$	
Toggle i-th bit	$x \wedge= (1 \ll i)$	$O(1)$	
Lowest set bit	$x \& -x$	$O(1)$	Value, not index. Fenwick tree core
Drop lowest set bit	$x \& (x - 1)$	$O(1)$	Kernighan's bit-count loop
Is power of two	$x \&\& !(x \& (x - 1))$	$O(1)$	Guard against $x = 0$
Count set bits	<code>__builtin_popcount(x)</code>	$O(1)$	<code>popcountll</code> for 64-bit
Trailing zeros	<code>__builtin_ctz(x)</code>	$O(1)$	= index of lowest set bit
Leading zeros	<code>__builtin_clz(x)</code>	$O(1)$	Undefined when $x = 0$
Highest bit index	$31 - \text{__builtin_clz}(x)$	$O(1)$	$\text{floor}(\log_2 x)$
Parity of set bits	<code>__builtin_parity(x)</code>	$O(1)$	1 if odd count
Iterate all masks	<code>for(int m=0; m<(1<<n); ++m)</code>	$O(2^n)$	Bitmask DP outer loop
Iterate submasks	<code>for(int s=m; s; s=(s-1)&m)</code>	$O(3^n)$ total	Skips $s = 0$; handle it separately
Swap without temp	$a \wedge= b; b \wedge= a; a \wedge= b;$	$O(1)$	Fails if a and b alias

15. bitset — Fixed-Size Bit Array

What it does	Memory	Does NOT support
N bits packed into $N/64$ machine words. Every whole-set operation runs 64 bits at a time, giving a constant-factor speedup of roughly $64\times$ over a bool array. Same asymptotics, far smaller runtime.	$N/8$ bytes	Size must be a compile-time constant — no resize. No <code>push_back</code> . <code>_Find_first</code> / <code>_Find_next</code> are GCC extensions, not standard.

```
#include <bitset>    bitset<1000> b;    // N must be a constant, not a variable
```

Operation	C++ Function	Time	Notes
Read / write bit	<code>b[i]</code>	$O(1)$	Returns a proxy reference
Set bit / all bits	<code>b.set(i)</code> / <code>b.set()</code>	$O(1)$ / $O(N/64)$	
Clear bit / all	<code>b.reset(i)</code> / <code>b.reset()</code>	$O(1)$ / $O(N/64)$	
Flip bit / all	<code>b.flip(i)</code> / <code>b.flip()</code>	$O(1)$ / $O(N/64)$	
Count set bits	<code>b.count()</code>	$O(N/64)$	Word-wise popcount
Any / none / all	<code>b.any()</code> / <code>b.none()</code> / <code>b.all()</code>	$O(N/64)$	
First set bit	<code>b._Find_first()</code>	$O(N/64)$	Returns N if empty (GCC only)
Next set bit	<code>b._Find_next(i)</code>	$O(N/64)$	Iterate a sparse set fast

Operation	C++ Function	Time	Notes
Bitwise ops	<code>b &= c / b = c / b ^= c</code>	$O(N/64)$	The whole point — 64 bits per step
Shift	<code>b <<= k / b >>= k</code>	$O(N/64)$	Subset-sum DP in one line
Convert out	<code>b.to_string() / b.to_ullong()</code>	$O(N)$	<code>to_ullong</code> throws if $N > 64$
Size	<code>b.size()</code>	$O(1)$	The compile-time N

When bitset wins: knapsack / subset-sum reachability (`dp |= dp << w;`), dense graph reachability and transitive closure, and any $O(n^2/64)$ sieve-like loop. It does not change the complexity class — it buys you roughly a 64× constant factor, which is often exactly the gap between TLE and AC.